

Guía de actividades



Universidad de
La Sabana

A continuación, le presentamos las orientaciones de las actividades de aprendizaje de la unidad de **Ecosistema DevOps: hHerramientas para CI/CD y monitoreo**.

Para alcanzar una comprensión integral del ecosistema DevOps, debemos familiarizarnos con tecnologías clave como Docker, Kubernetes y Jenkins, que permiten encapsular aplicaciones, orquestar contenedores y automatizar pipelines de CI/CD, respectivamente. Lograremos implementar prácticas de monitoreo continuo, logging efectivo y mecanismos de seguridad que garanticen la estabilidad, el rendimiento y la resiliencia de los sistemas en entornos productivos. Mediante actividades como laboratorios técnicos, simulaciones de incidentes, análisis post-mortem y ensayos argumentativos, se fortalecerán habilidades para diagnosticar, responder y planear acciones futuras ante desafíos operativos. El trabajo colaborativo será fundamental para construir soluciones sostenibles, reflexionar sobre su impacto y proyectar implementaciones para software que toman cada día más impacto en el entorno tecnológico como motores de machine Learning acompañados de otras tecnologías IA que para su mantenimiento sostenible usan bases robustas de DevOps para convertirlo el MLOps y Xops.

Generalidades

Tiempo total previsto para el desarrollo de las actividades: 13 horas

Resultados previstos de aprendizaje:

- Resolver problemas complejos de arquitectura de software, enfocándose en la optimización de diseño, la seguridad integral y la funcionalidad general utilizando técnicas avanzadas de análisis arquitectónico, pruebas sistemáticas y evaluación crítica para desarrollar y validar soluciones que sean robustas, escalables y eficientes.

Indicadores de desempeño

- Integrar componentes tecnológicos y organizacionales en soluciones DevOps, considerando sus interrelaciones, impactos y retroalimentaciones dentro del ciclo de vida del desarrollo de software.

Introducción al momento:

Esta etapa del proyecto está orientada a la implementación práctica de soluciones DevOps mediante el desarrollo de dos laboratorios técnicos. Aquí aplicarás los conocimientos adquiridos sobre integración continua (CI), entrega continua (CD), automatización, seguridad y monitoreo. El objetivo es que configures pipelines funcionales que reflejen eficiencia operativa, resiliencia técnica y alineación con estándares del sector. Esta fase representa el momento en que la teoría se transforma en acción, permitiéndote construir entornos tecnológicos robustos y sostenibles, mientras desarrollas habilidades clave para liderar procesos de cambio en equipos de desarrollo y operaciones.

Actividad**Actividad 1: Laboratorio técnico (Evaluativa).**

Tiempo estimado para su realización: 8 horas

Tipo de trabajo: individual

Descripción:

Implementar un pipeline CI/CD completo que integre prácticas de seguridad y monitoreo.

Introducción:

Los laboratorios técnicos es la manera en la que se ira construyendo la aplicación y así llega el momento en el que se pone en práctica lo aprendido. A lo largo del curso, el proyecto será el eje central de su proceso formativo, permitiéndole no solo la adquisición de conocimientos teóricos, sino también la transformación del conocimiento en acciones concretas para equipos técnicos y transformacionales DevOps. Debido a que el presente modulo está diseñado bajo la metodología de aprendizaje basada en proyectos (ABP), comprende tres fases la creación de la estructura de los pipelines de ci/cd para una aplicación con código alojado en github, posteriormente se hará la implementación real de la aplicación en la que se requerirá la construcción de la misma en un entorno k8s y la ejecución de conectores de seguridad y finalmente la habilitación de monitoreo a dicha aplicación.

Contexto:

Esta es el segundo entregable del proyecto, que corresponde a una fase de implementación en el que se crearan dos pipelines para lograr ci y cd de una aplicación web que tiene su código alojado en git y se requiere que se realice integración con herramientas de seguridad e integración de monitoreo, las herramientas a implementar son: Github Actions, Jenkins, sonarqube, Snyk, Prometheus, Grafana.

Objetivo del laboratorio

Implementar un pipeline CI/CD completo que integre prácticas de seguridad y monitoreo, fortaleciendo la eficiencia operativa y la cobertura del ciclo de vida de desarrollo de software.

- i. Pipeline CI/CD funcional (GitHub Actions + Jenkins).
- ii. Integración de herramientas de seguridad como:
 - a. SonarQube: análisis estático de código.
 - b. Snyk: detección de vulnerabilidades en dependencias.
- iii. Integración de monitoreo con:
 - a. Prometheus: recolección de métricas.
 - b. Grafana: visualización de métricas en dashboards.

Entregable

- Repositorio funcional en GitHub con:
 - Código fuente.
 - Configuración de CI (ci.yml) y CD (Jenkinsfile).
 - Archivos de despliegue para Kubernetes.
 - Configuración de seguridad y monitoreo.
- Dashboard de monitoreo activo:
 - Visualización de métricas clave (uso de CPU, memoria, estado de pods, etc.).
 - Capturas o enlace al dashboard en Grafana.
- Informe de seguridad:
 - Resultados del análisis de SonarQube y/o Snyk.
 - Identificación de vulnerabilidades y recomendaciones de mejora.

Instrucciones de entrega

- Enlace al repositorio de GitHub.
- Capturas de pantalla del pipeline en ejecución (CI y CD).

- Capturas del dashboard de Grafana.
- Capturas o archivo del informe de seguridad.
- Documento técnico (PDF o Markdown) que incluya:
 - Descripción del flujo CI/CD.
 - Herramientas utilizadas y justificación.
 - Evidencia de seguridad y monitoreo.
 - Reflexión sobre eficiencia operativa.

Recomendaciones para la realización

- Usar contenedores para herramientas como SonarQube, Prometheus y Grafana (Docker o Helm en Kubernetes).
- Configurar alertas básicas en Grafana si es posible.
- Documentar claramente cómo se integran las herramientas en el pipeline.
- Priorizar la automatización y la reutilización de configuraciones.

Rúbrica de evaluación

Criterios de evaluación	Indicadores de cumplimiento: Línea del tiempo									
	Excelente		Bueno		Necesita mejorar		Deficiente		No cumple con el criterio o no entrega	
	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem
Configura y ejecuta correctamente pipelines con GitHub Actions y Jenkins	1.0	Pipeline completo, funcional y automatizado.	0.8	Pipeline funcional con integración parcial.	0.6	Pipeline incompleto o con errores menores.	0.5	Pipeline con errores graves o sin ejecución.	0	No configura el pipeline.
Integra SonarQube y/o Snyk en el pipeline con resultados visibles	1.0	Integración completa con análisis y recomendaciones	0.8	Integración funcional con resultados básicos.	0.6	Integración parcial o sin evidencia clara.	0.5	Integración débil o sin resultados.	0	No integra herramientas de seguridad.
Configura Prometheus y Grafana con visualización de métricas	0.5	Dashboard activo con métricas clave y alertas básicas.	0.4	Dashboard funcional con métricas generales.	1.2	Dashboard incompleto o sin conexión clara.	1.0	Dashboard confuso o sin datos relevantes.	0	No configura monitoreo.
Explica el flujo CI/CD, herramientas, evidencias y reflexión	0.5	Documentación clara, estructurada y con reflexión profunda.	0.4	Documentación adecuada con explicación general.	0.3	Documentación incompleta o poco clara.	0.25	Documentación confusa o sin reflexión.	0	No entrega documentación.
Repositorio organizado, funcional y accesible	2.0	Repositorio completo con código,	1.6	Repositorio funcional pero con organización básica.	0.3	Repositorio incompleto desorganizado.	0.25	Repositorio con errores o inaccesible.	0	No entrega repositorio.

		configuraciones y evidencias.								
TOTAL	5.0		4.0		3.0		2.5		0.0	

Actividad 2: Informe: estudio de caso, enfoque estratégico (Evaluativa).

Tiempo estimado para su realización: 5 horas

Tipo de trabajo: Grupal (número de integrantes: 2, máximo 3 personas)

Descripción:

Simulación de incidente + análisis post-mortem.

Contexto:

Fomentar la reflexión profunda sobre el proceso de implementación del pipeline CI/CD, identificar oportunidades de mejora, y proyectar la evolución del enfoque DevOps hacia MLOps en contextos reales.

Recomendaciones para la realización:

- Documento tipo postmortem con hallazgos y recomendaciones. Asignen roles claros: redacción, investigación, análisis, edición.
 - ¿Qué salió bien?
 - ¿Qué salió mal?
 - ¿Qué aprendimos?
 - ¿Qué podemos mejorar?
- Mapa conceptual o presentación breve comparando DevOps y MLOps.
 - ¿Qué es MLOps y cómo se diferencia de DevOps?
 - ¿Qué nuevos retos aparecen al integrar modelos de machine learning?
 - ¿Cómo se adaptan los pipelines CI/CD para modelos entrenables?
 - ¿Qué herramientas emergen en MLOps? (Ej. MLflow, Kubeflow, DVC)
 - Referencias: incluyan las fuentes consultadas en formato APA.
- Uso de recursos: lean y citen los siguientes recursos:

- Google. (n.d.). *How SRE relates to other disciplines*.
<https://sre.google/workbook/how-sre-relates/>
- Spotify Engineering. (2020, August 25). *How we improved developer productivity for our DevOps teams*.
<https://engineering.atspotify.com/2020/08/how-we-improved-developer-productivity-for-our-devops-teams/>
- Estilo y profundidad: mantengan un tono formal, claro y coherente.
 - Sustenten sus afirmaciones con ejemplos concretos y referencias bibliográficas.
 - Eviten generalizaciones; enfoquen el análisis en el caso específico.
- Extensión: se recomienda una extensión de hasta 1000 palabras.

Referencias:

- Osipov, C. (2022). *MLOps engineering at scale*. Manning Publications.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
https://bibliosabana.primo.exlibrisgroup.com/permalink/57US_INST/1i0k585/cdi_askewsholts_vlebooks_9780321670274

Rúbrica de evaluación

Criterios de evaluación	Indicadores de cumplimiento: Línea del tiempo									
	Excelente		Bueno		Necesita mejorar		Deficiente		No cumple con el criterio o no entrega	
	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem
Análisis postmortem del incidente Identifica aciertos, errores, aprendizajes y mejoras	1.0	Análisis profundo, estructurado y con recomendaciones claras.	0.8	Análisis adecuado con reflexión general.	0.6	Análisis limitado o poco argumentado.	0.5	Análisis superficial o sin recomendaciones.	0	No entrega análisis postmortem.
Explica diferencias, retos y adaptación de pipelines, Comparación DevOps vs MLOps	1.0	Comparación clara, con ejemplos y herramientas emergentes.	0.8	Comparación adecuada con conceptos generales.	0.6	Comparación incompleta o poco clara.	0.5	Comparación débil o sin conexión técnica.	0	No realiza comparación.

Integra lecturas recomendadas y otras fuentes relevantes	0.5	Cita correctamente y relaciona conceptos con claridad.	0.4	Usa fuentes con conexión general.	0.3	Menciona fuentes sin integración clara.	0.25	Mención vaga o incorrecta.	0	No usa fuentes.
Representa visualmente la transición DevOps → MLOps	2.0	Mapa/presentación clara, estructurada y con contenido técnico.	1.6	Mapa/presentación adecuada con elementos clave.	1.2	Mapa/presentación incompleta o poco clara.	1	Mapa/presentación confusa o sin contenido técnico.	0	No entrega representación visual.
Claridad, estilo y profundidad del informe	0.5	Informe bien estructurado, claro y con profundidad conceptual.	0.4	Informe adecuado con buena organización.	0.3	Claridad, estilo y profundidad del informe.	0.25	Informe bien estructurado, claro y con profundidad conceptual.	0	Informe adecuado con buena organización.
TOTAL	5.0		4.0		3.0		2.5		0.0	